

Dokumentacja Snapper

Kompleksowy przewodnik po projekcie

Wygenerowano 2026-01-18

Spis treści

[Snapper](#)

[API](#)

[Architecture](#)

[CLI](#)

[Configuration](#)

[Development](#)

[Messaging \(ZeroMQ\)](#)

[Trading Strategies](#)

Snapper

Trading platform with market data collection, trading engine, and backtester. Supports Kraken, Zonda, Walumat exchanges and Polygon.io data.

Features

- **Market data collection** — WebSocket and REST API from Kraken, Zonda, Walumat, Polygon.io
- **Trading engine** — Order execution in live and paper trading modes
- **Strategies** — Framework for creating strategies based on RSI, MACD, cointegration, and TA-Lib indicators
- **ZeroMQ messaging** — Pub/sub architecture for market data and signals
- **Web dashboard** — FastAPI + React with real-time WebSocket
- **CLI** — Full system management via Typer CLI
- **Backtesting** — Strategy testing on historical data

Quick Start

Requirements

- Python 3.12+
- Poetry
- Node.js 20+ and pnpm (for frontend)
- TA-Lib (C library)

Installation

```
# Clone repository
git clone https://github.com/user/snapper.git
cd snapper

# Install system dependencies (macOS)
brew install ta-lib

# Install Python dependencies
make setup

# Install frontend dependencies
make ui-setup
```

```
# Copy and configure environment variables
cp .env.example .env
```

Configuration

Edit .env file:

```
# Database (SQLite by default, PostgreSQL optional)
DB_URL=sqlite+aiosqlite:///./data/snapper.db

# Settings encryption in database
MASTER_PASSWORD=your_master_password
ENCRYPTION_SALT=your_salt

# HTTP Server
SERVER_HOST=127.0.0.1
SERVER_PORT=8000

# ZeroMQ broker
ZMQ_BROKER_XSUB=tcp://127.0.0.1:7500
ZMQ_BROKER_XPUB=tcp://127.0.0.1:7501
```

Running

```
# Initialize database
snapper db-init

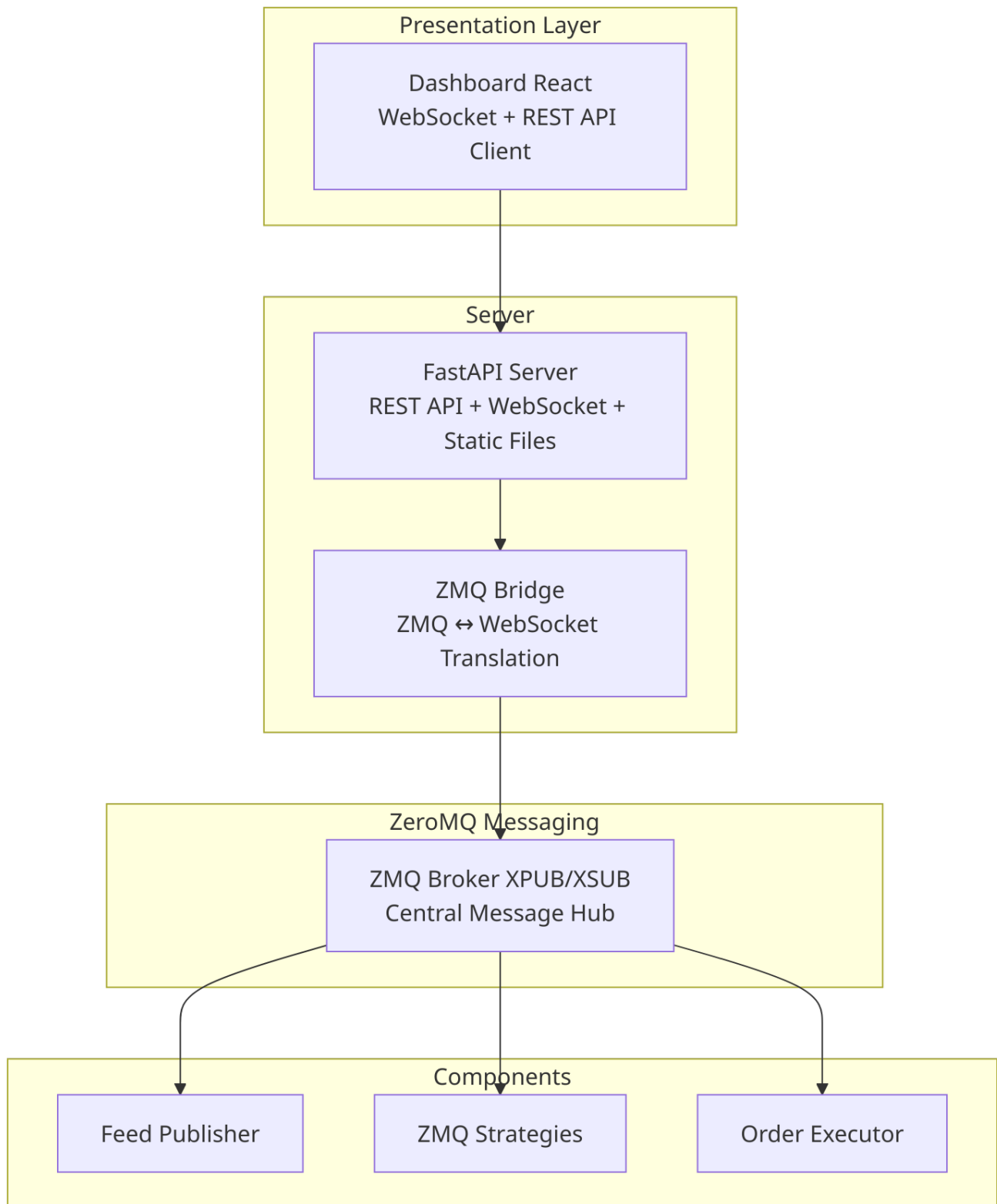
# Run migrations
snapper db-upgrade

# Create admin user
snapper init-admin

# Start server
snapper server
```

Dashboard available at <http://localhost:8000>.

Architecture



CLI Commands

Server and Infrastructure

```
snapper server          # Start FastAPI server
snapper broker          # Start ZMQ broker
snapper trade-zmq       # Trading coordinator
snapper executor        # Order executor
snapper feed            # Market data publisher
```

Database

```
snapper db-init         # Initialize schema
snapper db-upgrade      # Alembic migrations
snapper db-downgrade    # Rollback migrations
```

Users

```
snapper init-admin      # Create admin user
snapper list-users      # List users
snapper reset-password  # Reset password
```

Market Data

```
snapper update-kraken-symbols  # Sync Kraken symbols
snapper update-polygon-symbols # Sync Polygon symbols
snapper polygon-backfill-aggregates # Backfill historical data
```

Strategies

Creating your own strategy:

```
from snapper.strategies.base import BaseStrategy, Signal, StrategyConfig
from snapper.strategies.decorators import register_strategy,
create_strategy_process
from snapper.messaging.schemas.messages import BarEnvelope

@register_strategy("MyStrategy")
@create_strategy_process(
    process_name="my_strategy",
    default_config={
        "name": "my_strategy",
        "inputs": ["market.kraken.BTC-USD.candles.1h"],
        "outputs": ["BTC-USD"],
        "exchange": "paper",
        "params": {"threshold": 0.5},
    },
)
class MyStrategy(BaseStrategy):
```

```

def __init__(self, config: StrategyConfig) -> None:
    super().__init__(config)
    self.threshold = self.params.get("threshold", 0.5)

    async def on_bar(self, instrument: str, bar: BarEnvelope) -> Signal | None:
        # Strategy logic
        if some_condition:
            return Signal(
                instrument=instrument,
                side="buy",
                strength=1.0,
                price=bar.close,
                reason="Condition met",
            )
        return None

```

Development

Quality Gates

```

make check-all    # Full quality gate (backend + frontend + tests + 100%
coverage)
make fix-all       # Automatic formatting and linting fixes

```

Individual Steps

```

make fmt           # Check formatting
make lint          # Linting (ruff)
make typecheck     # Type checking (mypy)
make test          # Unit tests
make cov           # Tests with coverage (100% required)

```

Frontend

```

make ui-dev        # Vite development server
make ui-build      # Production build
make ui-lint       # ESLint
make ui-format     # Prettier

```

Docker

```

make docker-build  # Build image
make docker-run    # Run container
make docker-stop   # Stop container

```

Or with docker-compose:

```
docker-compose up -d
```

Documentation

Detailed documentation in [docs/](#) directory:

- [Architecture](#) — System structure and layers
- [Configuration](#) — Environment variables and settings
- [CLI](#) — Full command documentation
- [Strategies](#) — Creating trading strategies
- [API](#) — REST API and WebSocket
- [Messaging](#) — ZeroMQ architecture
- [Development](#) — Developer guidelines

License

MIT License — see [LICENSE](#) file.

API

Snapper provides a REST API and WebSocket interface for platform interaction. API requires JWT authentication via HTTP-only cookies.

Authentication

Authentication uses HTTP-only cookies for security. After login, the server sets `access_token`, `refresh_token`, and `csrf_token` cookies automatically.

Login

```
POST /api/auth/login
Content-Type: application/json

{
  "username": "admin",
  "password": "password123"
}
```

Response:

The server sets HTTP-only cookies and returns user info:

```
{
  "user": {
    "id": 1,
    "username": "admin",
    "role": "admin",
    "is_active": true
  }
}
```

Cookies set:

Cookie	HttpOnly	Secure	SameSite	Description
access_token	Yes	Yes (prod)	Lax/Strict	JWT access token (15min)
refresh_token	Yes	Yes (prod)	Lax/Strict	JWT refresh token (7 days)
csrf_token	No	Yes (prod)	Lax/Strict	CSRF protection token

Refresh Token

```
POST /api/auth/refresh
```

The refresh_token cookie is sent automatically by the browser. Server returns new tokens in cookies.

Logout

```
POST /api/auth/logout
X-CSRF-Token: <csrf_token>
```

Clears all authentication cookies.

CSRF Token

For mutating requests (POST, PUT, DELETE), a CSRF token is required:

```
GET /api/auth/csrf
```

Response:

```
{
  "csrf_token": "abc123..."
}
```

Then add the header to mutating requests:

```
X-CSRF-Token: abc123...
```

REST Endpoints

Health

```
GET /api/health
```

Response:

```
{
  "status": "healthy",
  "timestamp": "2026-01-18T12:00:00Z",
  "version": "0.1.0",
  "connections": {
    "active": 5,
  }
}
```

```
    "total": 100
  },
  "topics": {
    "available": 50,
    "active": 12
  }
}
```

Candles (OHLCV)

```
GET /api/candles?instrument=BTC-USD&timeframe=1h&limit=100
X-CSRF-Token: <csrf_token>
```

Parameters:

Parameter	Type	Required	Description
instrument	string	yes	Instrument symbol
timeframe	string	yes	Timeframe (1m, 5m, 15m, 1h, 4h, 1d)
limit	int	no	Number of candles (max 1000, default 100)

Response:

```
[
  {
    "instrument": "BTC-USD",
    "timeframe": "1h",
    "timestamp": "2026-01-18T11:00:00Z",
    "open": 42000.0,
    "high": 42500.0,
    "low": 41800.0,
    "close": 42300.0,
    "volume": 1234.56,
    "vwap": 42150.0,
    "trades": 5678
  }
]
```

Orders

```
GET /api/orders?symbol=BTC-USD&limit=100&offset=0
X-CSRF-Token: <csrf_token>
```

Parameters:

Parameter	Type	Required	Description
symbol	string	no	Filter by symbol

Parameter	Type	Required	Description
limit	int	no	Number of orders (max 1000, default 100)
offset	int	no	Skip N orders (default 0)

Response:

```
[
  {
    "id": 1,
    "instrument": "BTC-USD",
    "exchange": "kraken",
    "client_order_id": "ord_123",
    "exchange_order_id": "KRAKEN-456",
    "created_at": "2026-01-18T12:00:00Z",
    "updated_at": "2026-01-18T12:01:00Z",
    "side": "buy",
    "type": "limit",
    "price": 42000.0,
    "size": 0.1,
    "status": "filled",
    "time_in_force": "GTC",
    "error": null
  }
]
```

Signals

```
GET /api/signals?instrument=BTC-USD&strategy=rsi&hours=24&limit=100
X-CSRF-Token: <csrf_token>
```

Parameters:

Parameter	Type	Required	Description
instrument	string	no	Filter by instrument
strategy	string	no	Filter by strategy
hours	int	no	History hours (max 168, default 24)
limit	int	no	Number of signals (max 1000, default 100)

Response:

```
[
  {
    "id": 1,
    "instrument": "BTC-USD",
    "exchange": "paper",
    "timestamp": "2026-01-18T12:00:00Z",
  }
]
```

```
    "side": "buy",
    "strength": 0.85,
    "reason": "RSI 28.5 <= 30",
    "strategy_name": "rsi_btc_1h",
    "price": 42000.0
  }
]
```

Executions

```
GET /api/executions?limit=100
X-CSRF-Token: <csrf_token>
```

Response:

```
[
  {
    "id": 1,
    "order_id": 123,
    "timestamp": "2026-01-18T12:01:00Z",
    "price": 42000.0,
    "size": 0.1,
    "fee": 0.001,
    "fee_asset": "USD",
    "instrument": "BTC-USD",
    "side": "buy",
    "exchange": "kraken"
  }
]
```

Positions

```
GET /api/positions
X-CSRF-Token: <csrf_token>
```

Response:

```
[
  {
    "id": 1,
    "instrument": "BTC-USD",
    "exchange": "kraken",
    "quantity": 0.5,
    "average_price": 41500.0,
    "unrealized_pnl": 250.0,
    "realized_pnl": 100.0,
    "updated_at": "2026-01-18T12:00:00Z"
  }
]
```

WebSocket Stats

```
GET /api/ws/stats
X-CSRF-Token: <csrf_token>
```

Response:

```
{
  "websocket": {
    "active_connections": 5,
    "topic_subscribers": {
      "market.kraken.BTC-USD.candles.1h": 3,
      "signals.paper.BTC-USD": 2
    },
    "client_count": 5
  },
  "zmq_bridge": {
    "active_topics": 12,
    "subscriber_tasks": 12,
    "available_topics": ["market.kraken.BTC-USD.candles.1h", "..."]
  },
  "config": {
    "max_connections": 100,
    "heartbeat_interval": 30
  }
}
```

ZMQ Health

```
GET /api/zmq/health
X-CSRF-Token: <csrf_token>
```

Response:

```
{
  "status": "healthy",
  "components": {
    "broker": "running",
    "bridge": "running"
  },
  "config": {
    "xsub_endpoint": "tcp://127.0.0.1:7500",
    "xpub_endpoint": "tcp://127.0.0.1:7501"
  }
}
```

Process Management

List Processes

```
GET /api/processes
```

Response:

```
[
  {
    "name": "zmq_broker",
    "description": "ZeroMQ XPUB/XSUB message broker",
    "status": "running",
    "role": "core",
    "priority": 10,
    "enabled": true,
    "pid": 12345,
    "started_at": "2026-01-18T10:00:00Z"
  },
  {
    "name": "strategy_rsi_btc_1h",
    "description": "RSI Reversion Strategy for BTC",
    "status": "stopped",
    "role": "strategy",
    "priority": 50,
    "enabled": false
  }
]
```

Start Process

```
POST /api/processes/{name}/start
X-CSRF-Token: <csrf_token>
```

Stop Process

```
POST /api/processes/{name}/stop
X-CSRF-Token: <csrf_token>
```

System Status

```
GET /api/system/status
```

Response:

```
{
  "running_processes": 5,
  "total_processes": 10,
  "cpu_usage": 25.5,
  "memory_usage": 512000000,
```

```
}  
  "uptime_seconds": 3600  
}
```

Settings API

List Settings

```
GET /api/settings
```

Get Setting

```
GET /api/settings/{key}
```

Save Setting

```
PUT /api/settings/{key}  
X-CSRF-Token: <csrf_token>  
Content-Type: application/json  
  
{  
  "value": "new_value",  
  "encrypted": true  
}
```

WebSocket

Connection

WebSocket endpoint requires a one-time token for authentication:

```
ws://localhost:8000/api/ws?token=<ws_token>
```

Get the WebSocket token from:

```
GET /api/auth/ws-token
```

Message Protocol

All messages are in JSON format.

Subscribe

```
{
  "type": "subscribe",
  "topics": ["market.kraken.BTC-USD.candles.1h", "signals.paper.BTC-USD"]
}
```

Unsubscribe

```
{
  "type": "unsubscribe",
  "topics": ["market.kraken.BTC-USD.candles.1h"]
}
```

Ping/Pong

```
{
  "type": "ping"
}
```

Response:

```
{
  "type": "pong",
  "timestamp": 1705579200000
}
```

Server Messages

Market Data (Bar)

```
{
  "type": "bar",
  "topic": "market.kraken.BTC-USD.candles.1h",
  "data": {
    "open": 42000.0,
    "high": 42500.0,
    "low": 41800.0,
    "close": 42300.0,
    "volume": 1234.56,
    "timestamp": 1705579200,
    "timeframe": "1h",
    "instrument": "BTC-USD"
  }
}
```

Tick

```
{
  "type": "tick",
  "topic": "market.kraken.BTC-USD.ticks",
  "data": {
    "price": 42150.0,
    "volume": 0.5,
    "timestamp": 1705579200123,
    "instrument": "BTC-USD"
  }
}
```

Signal

```
{
  "type": "signal",
  "topic": "signals.paper.BTC-USD.rsi_btc_1h",
  "data": {
    "instrument": "BTC-USD",
    "side": "buy",
    "strength": 0.85,
    "reason": "RSI 28.5 <= 30",
    "price": 42000.0,
    "timestamp": 1705579200,
    "metadata": {
      "rsi_value": 28.5,
      "period": 14
    }
  }
}
```

Trade Execution

```
{
  "type": "trade",
  "topic": "fills.paper.BTC-USD",
  "data": {
    "order_id": "ord_123",
    "instrument": "BTC-USD",
    "side": "buy",
    "price": 42000.0,
    "size": 0.1,
    "fee": 0.001,
    "timestamp": 1705579200
  }
}
```

Heartbeat

```
{
  "type": "heartbeat",
  "topic": "system.heartbeat",
}
```

```
"data": {
  "component": "zmq_broker",
  "status": "healthy",
  "timestamp": 1705579200
}
```

Error

```
{
  "type": "error",
  "code": "INVALID_TOPIC",
  "message": "Topic 'invalid.topic' does not exist"
}
```

Available Topics

Market Data

- `market.{exchange}.{instrument}.candles.{timeframe}` — OHLCV candles
- `market.{exchange}.{instrument}.ticks` — Price ticks

Signals

- `signals.paper.{instrument}.{strategy}` — Paper trading signals
- `signals.{exchange}.{instrument}.live` — Live signals

Fills

- `fills.{exchange}.{instrument}` — Order executions

System

- `system.heartbeat` — Component heartbeats
- `system.process.{name}` — Process status

Error Codes

Code	HTTP Status	Description
UNAUTHORIZED	401	Missing or invalid token
FORBIDDEN	403	Insufficient permissions
CSRF_INVALID	403	Invalid CSRF token
NOT_FOUND	404	Resource not found

Code	HTTP Status	Description
VALIDATION_ERROR	422	Data validation error
INTERNAL_ERROR	500	Server error

Usage Example (Python)

```
import httpx

BASE_URL = "http://localhost:8000/api"

async def main():
    async with httpx.AsyncClient() as client:
        # Login - cookies are set automatically
        response = await client.post(
            f"{BASE_URL}/auth/login",
            json={"username": "admin", "password": "password123"}
        )
        # Cookies are stored in client automatically

        # Get CSRF token from cookie
        csrf_token = client.cookies.get("csrf_token")

        # Fetch candles - cookies sent automatically
        response = await client.get(
            f"{BASE_URL}/candles",
            params={"instrument": "BTC-USD", "timeframe": "1h"},
            headers={"X-CSRF-Token": csrf_token}
        )
        candles = response.json()
        print(candles)
```

WebSocket Example (JavaScript)

```
// Get WS token first (requires authenticated session)
const tokenResponse = await fetch('/api/auth/ws-token', {
  credentials: 'include' // Include cookies
});
const { token } = await tokenResponse.json();

const ws = new WebSocket(`ws://localhost:8000/api/ws?token=${token}`);

ws.onopen = () => {
  // Subscribe to topics
  ws.send(JSON.stringify({
    type: 'subscribe',
    topics: ['market.kraken.BTC-USD.candles.1h', 'signals.paper.BTC-USD']
  }));
};

ws.onmessage = (event) => {
```

```
    const message = JSON.parse(event.data);
    console.log('Received:', message.type, message.data);
};

// Heartbeat
setInterval(() => {
    ws.send(JSON.stringify({ type: 'ping' }));
}, 30000);
```

Architecture

Snapper is a trading platform built with a layered architecture using asynchronous processing and ZeroMQ messaging.

System Layers

Presentation Layer

Dashboard React +
FastAPI REST/WebSocket



Application Layer

Strategies, Trader
Coordinator, Process
Manager



Infrastructure Layer

ZMQ Messaging,
Exchange Clients, Data
Providers

Components

Core (src/snapper/core/)

Fundamental types and aliases used throughout the application:

- TradeSide — Trade direction (buy, sell)
- OrderType — Order type (market, limit, stop, stop_limit)
- OrderStatus — Order status in lifecycle
- ExecutionMode — Execution mode (live, paper)
- TradingExchange — Supported exchanges (kraken, zonda, walutomat, paper)

Config (src/snapper/config/)

Two-layer configuration:

1. **Bootstrap Settings** (bootstrap.py)

Settings from environment variables required before database connection:

- Database URL
- Master password for encryption
- HTTP and ZMQ server endpoints

2. **App Settings** (app.py)

Settings from database (encrypted):

- Exchange API keys
- Trading parameters
- Authentication configuration

Data (src/snapper/data/)

Persistence layer with SQLAlchemy:

- **ORM Models** (models.py):
 - Instrument — Financial instruments
 - Candle — OHLCV candles
 - Trade — Transactions
 - OrderRecord — Order history
 - Execution — Order executions
 - Position — Portfolio positions
 - StrategyRun — Strategy runs

- `SignalEvent` — Signal events
- `User` — System users
- `Setting` — Settings (encrypted)
- `SymbolMapping` — Symbol mappings between exchanges
- **Repository** (`repository.py`) — Async CRUD operations

Strategies (`src/snapper/strategies/`)

Trading strategies framework:

- **BaseStrategy** (`base.py`) — Base class for all strategies
- **Signal** — Trading signal structure
- **StrategyConfig** — Strategy configuration

Built-in strategies:

- `RSIReversion` (`rsi.py`) — Mean reversion on RSI
- `MACDCrossover` (`macd.py`) — MACD crossover
- `Cointegration` (`cointegration.py`) — Pairs trading

Indicators (`src/snapper/indicators/`)

Technical indicators:

- `rsi` — Relative Strength Index (Python implementation)
- `macd` — Moving Average Convergence Divergence
- `ta_lib_adapter` — TA-Lib library adapter

Messaging (`src/snapper/messaging/`)

ZeroMQ pub/sub architecture:



Components:

- **Broker** (`infrastructure/broker.py`) — XPUB/XSUB proxy
- **Publishers** (`publishers/`) — Market data publication
- **Executors** (`executors/`) — Order execution on exchanges
- **Schemas** (`schemas/`) — Pydantic message models
- **Topics** (`topics/`) — ZMQ topic definitions

Message types:

- TickEnvelope — Price tick
- BarEnvelope — OHLCV candle
- SignalEnvelope — Trading signal
- TradeEnvelope — Trade execution
- HeartbeatEnvelope — Component heartbeat

Application (src/snapper/application/)

Business logic:

- **Engine** (engine/) — Trader Coordinator
- **Process Manager** (process_manager/) — Process management
- **Services** (services/) — Application services
- **Updaters** (updaters/) — Data updates (symbols, historical)
- **Risk** (risk/) — Risk management
- **Portfolio** (portfolio/) — Portfolio management

Infrastructure (src/snapper/infrastructure/)

External integrations:

- **Exchanges** (exchanges/) — Exchange clients (Kraken, Zonda, Walutomat)
- **Market Data** (market_data/) — WebSocket feeds
- **Symbols** (symbols/) — Symbol mapping
- **Security** (security/) — Settings encryption
- **Logging** (logging/) — Logging configuration

Server (src/snapper/server/)

FastAPI application:

- **App Factory** (app.py) — create_app() with lifespan management
- **REST API** — HTTP endpoints
- **WebSocket** — Real-time streaming via ZMQ bridge
- **Static Files** — Frontend dashboard

API (src/snapper/api/)

API schemas:

- **Schemas** (schemas/) — Pydantic request/response models
- **Auth** (auth/) — Authentication services

Auth (src/snapper/auth/)

Authentication system:

- JWT tokens with access/refresh via HTTP-only cookies
- CSRF protection
- Role-based access (admin, user, viewer)
- WebSocket authentication

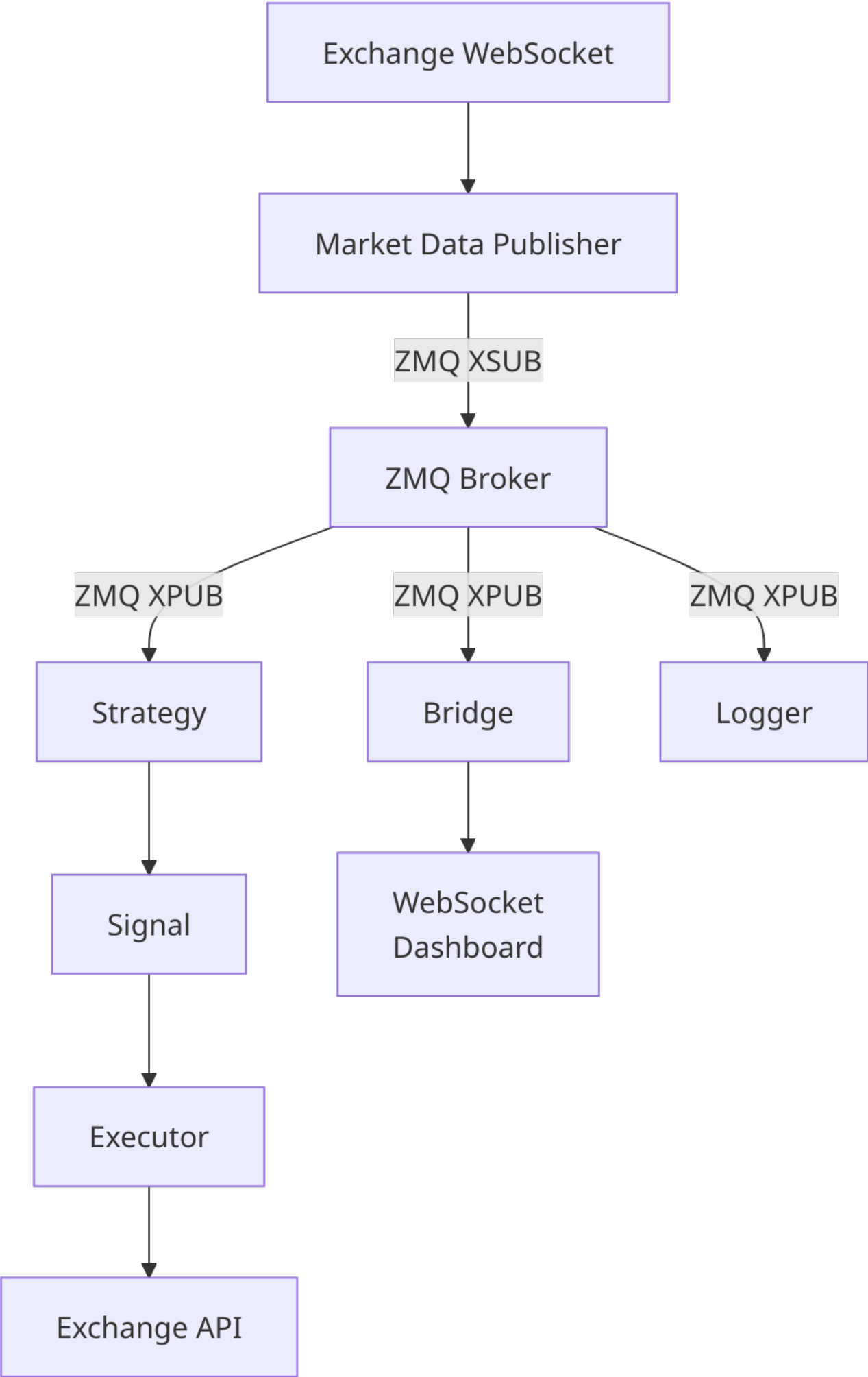
CLI (src/snapper/cli/)

Command-line interface (Typer):

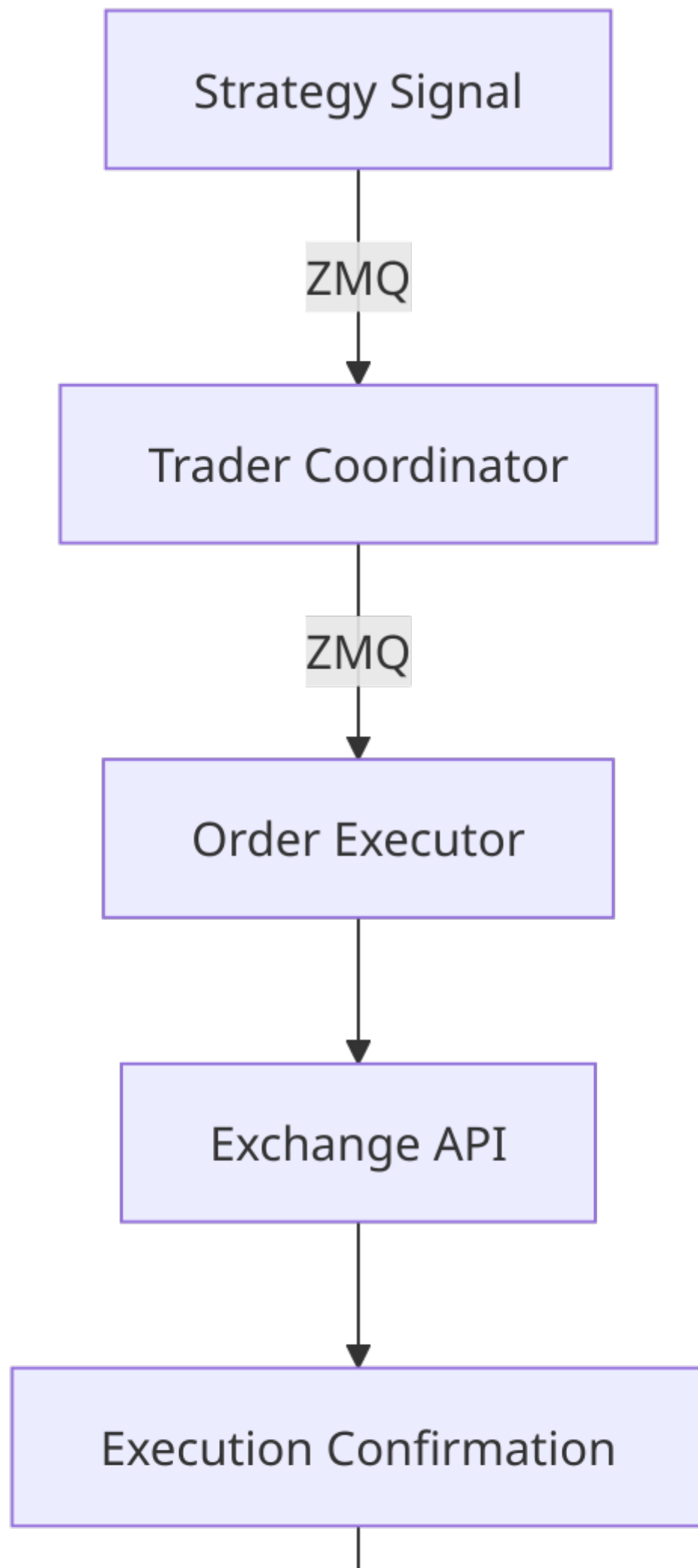
- Server and infrastructure management
- Database operations
- User management
- Market data updates

Data Flow

Market Data Flow



Order Flow



Database

SQLite by default, with PostgreSQL and Azure SQL support.

Schema

```
-- Main tables
instruments    -- Financial instruments
candles        -- OHLCV data
trades         -- Transaction history
order_records  -- Orders
executions     -- Executions
positions      -- Positions

-- Strategies
strategy_runs  -- Strategy runs
signal_events  -- Signals

-- System
users          -- Users
settings       -- Settings (encrypted)
symbol_mappings -- Symbol mappings
process_runs   -- Process history
```

Migrations

Alembic manages migrations:

```
snapper db-upgrade    # Apply migrations
snapper db-downgrade  # Rollback
```

Processes

The system manages processes through Process Manager:

- **Core** — Broker, Feed, Bridge (required)
- **Strategy** — Trading strategies
- **Task** — One-time tasks
- **Backtest** — Backtesting processes

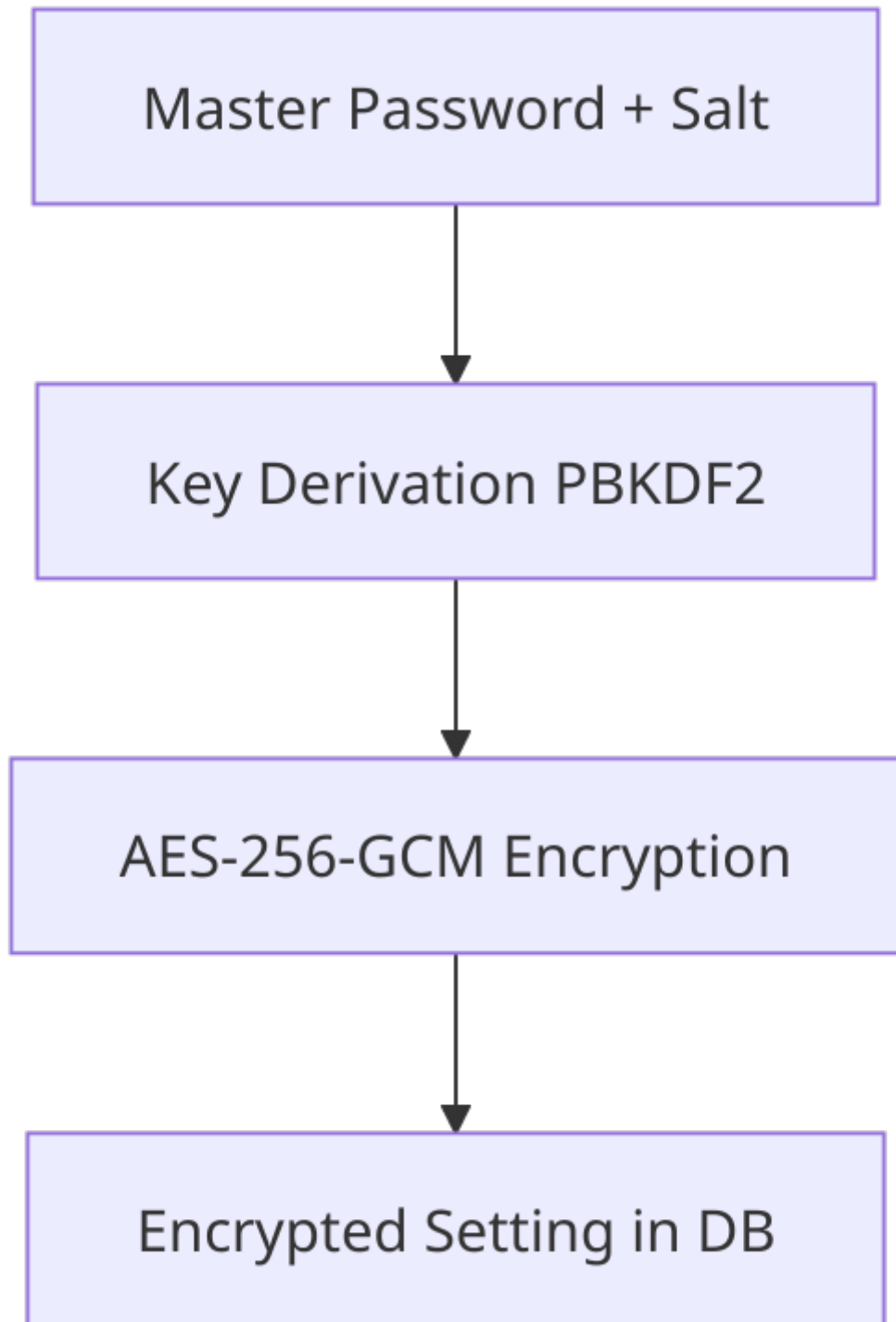
Processes can be:

- **long_running** — Run continuously (services)
- **one_shot** — Execute once (tasks)

Security

Settings Encryption

Sensitive data (API keys) are encrypted in the database:



Authentication

- JWT tokens via HTTP-only cookies (access: 15min, refresh: 7 days)
- CSRF tokens for mutating requests
- WebSocket tokens (one-time)
- Bcrypt password hashing

CLI

Snapper provides a command-line interface built on the Typer framework. All commands are available through the `snapper` command.

Usage

```
snapper [COMMAND] [OPTIONS]
```

Help for a specific command:

```
snapper [COMMAND] --help
```

Server and Infrastructure

server

Starts the FastAPI server with the web dashboard.

```
snapper server [OPTIONS]
```

Options:

Option	Type	Default	Description
--host	string	from env	Listen address
--port	int	from env	Server port
--reload	bool	from env	Auto-reload for development

Examples:

```
# Start with default settings
snapper server

# Start on all interfaces
snapper server --host 0.0.0.0 --port 8000

# Development mode with auto-reload
snapper server --reload
```

broker

Starts the ZeroMQ XPUB/XSUB broker for message routing.

```
snapper broker [OPTIONS]
```

Options:

Option	Type	Default	Description
--xsub	string	from env	XSUB endpoint (publishers)
--xpub	string	from env	XPUB endpoint (subscribers)

Example:

```
snapper broker --xsub tcp://127.0.0.1:7500 --xpub tcp://127.0.0.1:7501
```

trade-zmq

Starts the central trading coordinator.

```
snapper trade-zmq [OPTIONS]
```

Options:

Option	Type	Default	Description
--signal-topics	string	signals.	Signal topics to subscribe

Example:

```
snapper trade-zmq --signal-topics "signals.rsi,signals.macd"
```

executor

Starts the order execution service for a selected exchange.

```
snapper executor [OPTIONS]
```

Options:

Option	Type	Default	Description
-e, --exchange	string	kraken	Exchange (kraken, zonda, walutomat)

Examples:

```
# Kraken executor
snapper executor -e kraken

# Zonda executor
snapper executor --exchange zonda
```

feed

Starts the market data publisher via WebSocket.

```
snapper feed [OPTIONS]
```

Options:

Option	Type	Default	Description
--symbols	string	BTC/USD	Comma-separated symbols

Example:

```
snapper feed --symbols "BTC/USD,ETH/USD,SOL/USD"
```

zmq-logger

Monitors and logs ZMQ message traffic.

```
snapper zmq-logger [OPTIONS]
```

Options:

Option	Type	Default	Description
--payload/--no-payload	bool	false	Log full payload
--file/--no-file	bool	true	Write to audit file
--audit-file	string	data/zmq_audit.jsonl	Audit file path
--max-length	int	200	Max payload preview length

Example:

```
snapper zmq-logger --payload --audit-file logs/zmq.jsonl
```

Database

db-init

Initializes the database schema.

```
snapper db-init
```

db-upgrade

Runs Alembic migrations to a specified revision.

```
snapper db-upgrade [OPTIONS]
```

Options:

Option	Type	Default	Description
--revision	string	head	Target revision

Examples:

```
# Migrate to latest version
snapper db-upgrade

# Migrate to specific revision
snapper db-upgrade --revision abc123
```

db-downgrade

Rolls back Alembic migrations.

```
snapper db-downgrade [OPTIONS]
```

Options:

Option	Type	Default	Description
--revision	string	-1	Steps to roll back

Examples:

```
# Roll back one migration
snapper db-downgrade
```

```
# Roll back to specific revision
snapper db-downgrade --revision base
```

Users

init-admin

Creates the initial administrator user.

```
snapper init-admin [OPTIONS]
```

Options:

Option	Type	Default	Description
--username	string	admin	Username
--password	string	AdminSnapper2026!	Password

Example:

```
snapper init-admin --username admin --password SuperSecurePass123!
```

list-users

Displays a list of all system users.

```
snapper list-users
```

reset-password

Resets a user's password.

```
snapper reset-password USERNAME [OPTIONS]
```

Arguments:

- USERNAME — Username (required)

Options:

Option	Type	Default	Description
--new-password	string	(prompt)	New password

Example:

```
snapper reset-password john --new-password NewSecurePass123!
```

Symbol Synchronization

update-kraken-symbols

Synchronizes symbol mappings from the Kraken API.

```
snapper update-kraken-symbols [OPTIONS]
```

Options:

Option	Type	Default	Description
-f, --force	bool	false	Force update

update-zonda-symbols

Synchronizes symbol mappings from the Zonda API.

```
snapper update-zonda-symbols [OPTIONS]
```

Options:

Option	Type	Default	Description
-f, --force	bool	false	Force update

update-walutomat-symbols

Synchronizes symbol mappings from the Walutomat API.

```
snapper update-walutomat-symbols [OPTIONS]
```

Options:

Option	Type	Default	Description
-f, --force	bool	false	Force update

update-polygon-symbols

Synchronizes symbol mappings from the Polygon.io API.

```
snapper update-polygon-symbols [OPTIONS]
```

Options:

Option	Type	Default	Description
-f, --force	bool	false	Force update
--insert-new	bool	false	Insert new symbols

Note: This operation may take 10-15 minutes (44k+ symbols).

Market Snapshots

update-kraken-market-snapshot

Updates Kraken market snapshots with current prices.

```
snapper update-kraken-market-snapshot
```

update-zonda-market-snapshot

Updates Zonda market snapshots with current prices.

```
snapper update-zonda-market-snapshot
```

update-walutomat-market-snapshot

Updates Walutomat market snapshots with current prices.

```
snapper update-walutomat-market-snapshot
```

Data Backfill

polygon-backfill-aggregates

Fetches historical aggregated data from the Polygon.io API.

```
snapper polygon-backfill-aggregates [OPTIONS]
```

Options:

Option	Type	Default	Description
-s, --symbol	string[]	from settings	Symbols to backfill
--all	bool	false	All mapped symbols
-m, --multiplier	int	1	Timeframe multiplier
-t, --timespan	string	minute	Timespan (minute, hour, day)
-d, --days	int	from settings	Days back
--resume/--no-resume	bool	true	Resume from last timestamp
--csv/--no-csv	bool	true	Save to CSV.gz files

Examples:

```
# Backfill for specific symbols
snapper polygon-backfill-aggregates -s AAPL -s MSFT --days 365

# Backfill all mapped symbols
snapper polygon-backfill-aggregates --all --timespan day

# Hourly backfill with resume
snapper polygon-backfill-aggregates -s SPY -t hour -m 1 --resume
```

polygon-backfill-grouped

Fetches grouped daily data from the Polygon.io API.

```
snapper polygon-backfill-grouped [OPTIONS]
```

Encryption Management

settings-rotate-encryption

Rotates encryption keys for settings in the database.

```
snapper settings-rotate-encryption [OPTIONS]
```

Options:

Option	Type	Default	Description
--new-password	string	(required)	New master password
--new-salt	string	(unchanged)	New salt
--old-password	string	from env	Current password
--old-salt	string	from env	Current salt
--dry-run	bool	false	Show changes without applying

Example:

```
# Simulate rotation
snapper settings-rotate-encryption --new-password NewSecurePass --dry-run

# Actual rotation
snapper settings-rotate-encryption --new-password NewSecurePass --new-salt
NewSalt123
```

Example Workflows

New Installation Initialization

```
# 1. Initialize database
snapper db-init

# 2. Create administrator
snapper init-admin --username admin --password SecretPassword123!

# 3. Synchronize symbols
snapper update-kraken-symbols
snapper update-polygon-symbols

# 4. Start server
snapper server
```

Full Trading System Startup

```
# Terminal 1: ZMQ Broker
snapper broker

# Terminal 2: Data feed
snapper feed --symbols "BTC/USD,ETH/USD"

# Terminal 3: Executor
snapper executor -e kraken

# Terminal 4: Coordinator
```

```
snapper trade-zmq
```

```
# Terminal 5: Server with dashboard  
snapper server
```

Historical Data Backfill

```
# Backfill last 30 days for SPY  
snapper polygon-backfill-aggregates -s SPY --days 30 --timespan minute  
  
# Backfill daily data for all symbols  
snapper polygon-backfill-aggregates --all --timespan day --days 365
```

Configuration

Snapper's configuration system is based on two layers: environment variables (bootstrap) and database settings.

Environment Variables

Settings loaded from .env file or environment variables.

Database

Variable	Default	Description
DB_URL	sqlite+aiosqlite:///./data/snapper.db	SQLAlchemy connection URL

Database URL examples:

```
# SQLite (default)
DB_URL=sqlite+aiosqlite:///./data/snapper.db

# PostgreSQL
DB_URL=postgresql+asyncpg://user:password@localhost:5432/snapper

# Azure SQL
DB_URL=mssql+pyodbc://user:password@server.database.windows.net/snapper?
driver=ODBC+Driver+18+for+SQL+Server
```

Encryption

Variable	Default	Description
MASTER_PASSWORD	snapper_default_master_password_v1	Password for encrypting settings in database
ENCRYPTION_SALT	snapper_settings_salt_v1	Salt for key derivation

Important: Change default values in production environment.

HTTP Server

Variable	Default	Description
SERVER_HOST	127.0.0.1	Listen address

Variable	Default	Description
SERVER_PORT	8000	Server port
SERVER_RELOAD	false	Auto-reload for development

ZeroMQ

Variable	Default	Description
ZMQ_BROKER_XSUB	tcp://127.0.0.1:7500	XSUB endpoint (publishers connect here)
ZMQ_BROKER_XPUB	tcp://127.0.0.1:7501	XPUB endpoint (subscribers connect here)

Database Settings

Sensitive data is stored encrypted in the settings table.

Exchange API Keys

Key	Description
kraken_api_key	Kraken API key
kraken_api_secret	Kraken API secret
polygon_api_key	Polygon.io API key
walumat_api_key	Walumat API key
walumat_private_key	Walumat private key
zonda_api_key	Zonda API key
zonda_api_secret	Zonda API secret

Trading Parameters

Key	Default	Description
trading_instruments	["BTC-USD", "ETH-USD"]	List of trading instruments
default_timeframe	1h	Default candle timeframe
max_position_size	1.0	Maximum position size

Key	Default	Description
risk_per_trade	0.02	Risk per trade (2%)

Authentication

Key	Default	Description
access_token_expire_minutes	15	Access token lifetime
refresh_token_expire_days	7	Refresh token lifetime
csrf_token_expire_minutes	60	CSRF token lifetime

.env.example File

```
# =====
# Snapper Environment Configuration
# =====

# Database (SQLite default, PostgreSQL optional)
DB_URL=sqlite+aiosqlite:///./data/snapper.db

# Settings Encryption (CHANGE IN PRODUCTION!)
MASTER_PASSWORD=your_secure_master_password
ENCRYPTION_SALT=your_unique_salt_value

# HTTP Server
SERVER_HOST=127.0.0.1
SERVER_PORT=8000
SERVER_RELOAD=false

# ZeroMQ Broker Endpoints
ZMQ_BROKER_XSUB=tcp://127.0.0.1:7500
ZMQ_BROKER_XPUB=tcp://127.0.0.1:7501
```

Accessing Configuration in Code

Bootstrap settings (without database)

```
from snapper.config.settings import get_settings

settings = get_settings()
db_url = settings.db_url
host = settings.server_host
```

With database access

```
from snapper.config.settings import get_settings_with_service

settings = get_settings_with_service(settings_service)
api_key = settings.kraken_api_key # Decrypted from database
```

Managing Settings via API

Get Setting

```
GET /api/settings/{key}
```

Save Setting

```
PUT /api/settings/{key}
X-CSRF-Token: <csrf_token>
Content-Type: application/json

{
  "value": "new_value",
  "encrypted": true
}
```

List Settings

```
GET /api/settings
```

Environment Configuration

Development

```
# .env.development
DB_URL=sqlite+aiosqlite:///./data/snapper_dev.db
SERVER_HOST=127.0.0.1
SERVER_PORT=8000
SERVER_RELOAD=true
```

Production

```
# .env.production
DB_URL=postgresql+asyncpg://user:pass@db.example.com/snapper
MASTER_PASSWORD=<strong_random_password>
ENCRYPTION_SALT=<unique_random_salt>
SERVER_HOST=0.0.0.0
SERVER_PORT=8000
SERVER_RELOAD=false
```

Docker

Variables can be passed via docker-compose.yml:

```
services:
  snapper:
    env_file: .env
    environment:
      - DB_URL=postgresql+asyncpg://snapper:password@postgres/snapper
```

Settings Encryption

Sensitive settings are encrypted with AES-256-GCM:

1. Key is generated from MASTER_PASSWORD and ENCRYPTION_SALT (PBKDF2)
2. Value is encrypted with random IV
3. Ciphertext + IV + tag are stored in database as base64

If MASTER_PASSWORD changes, encrypted settings become inaccessible. Export settings before changing the password.

Configuration Validation

The system validates configuration at startup:

- Database connection check
- ZMQ endpoint verification
- Settings decryption test

Configuration errors are reported through exceptions with clear messages.

Development

Guidelines for developers working on the Snapper project.

Requirements

- Python 3.12+
- Poetry
- Node.js 20+ and pnpm
- TA-Lib (C library)
- Pre-commit hooks

Environment Setup

```
# Install system dependencies (macOS)
make system-deps

# Install Python dependencies
make setup

# Install frontend dependencies
make ui-setup

# Synchronize pre-commit hooks
make pre-refresh
```

Quality Gates

Main Command

```
make check-all
```

Executes the complete quality gate:

1. Formatting (ruff, black, isort)
2. Linting (ruff)
3. Type checking (mypy)
4. Docstring compliance
5. No # comments
6. No pragma/noqa/ignore

7. Tests with 100% coverage
8. Frontend (ESLint, Prettier, dead code)

Individual Steps

Command	Description
make fmt	Check formatting
make fmt-fix	Fix formatting
make lint	Ruff linting
make lint-fix	Fix linting issues
make typecheck	Mypy type checking
make test	Tests (parallel)
make test-serial	Tests (sequential, debug)
make cov	Tests with coverage

Automatic Fixes

```
make fix          # Backend: fmt-fix + lint-fix + move-imports
make fix-all      # Backend + frontend
```

Code Standards

Types (MANDATORY)

All functions must have type annotations:

```
def calculate_rsi(prices: pd.Series, period: int = 14) -> pd.Series:
    """Calculate RSI indicator."""
    ...

async def fetch_data(symbol: str) -> list[Candle]:
    """Fetch candle data."""
    ...
```

Test functions must have -> None:

```
async def test_rsi_calculation() -> None:
    """Test RSI calculation returns expected values."""
    ...
```

Docstrings (Google style)

```
def process_signal(signal: Signal, config: dict[str, Any]) -> Order | None:
    """Process trading signal and generate order.

    Takes a trading signal and configuration, validates the signal,
    and generates an appropriate order if conditions are met.

    Args:
        signal: Trading signal to process.
        config: Strategy configuration dictionary.

    Returns:
        Generated order if valid, None otherwise.

    Raises:
        ValueError: If signal data is invalid.
    """
    ...
```

No # Comments

comments are forbidden. Use docstrings:

```
# WRONG
def foo():
    # This calculates something
    x = 1 + 1

# CORRECT
def foo():
    """Calculate the sum."""
    x = 1 + 1
```

Imports

Imports at the top of file, single lines:

```
from datetime import UTC
from datetime import datetime
from typing import Any

import pandas as pd
from sqlalchemy import select
```

```
from snapper.config.settings import get_settings
from snapper.data.models import Candle
```

Forbidden:

- from __future__ import annotations
- if TYPE_CHECKING: blocks
- Re-exports in __init__.py

__init__.py

__init__.py files must be empty (or docstring only):

```
"""Snapper strategies module."""
```

Tests

Structure

```
tests/
├── conftest.py           # Shared fixtures
├── api/                  # API tests
├── application/          # Business logic tests
├── cli/                  # CLI tests
├── data/                 # Data layer tests
├── indicators/           # Indicator tests
├── messaging/            # ZMQ tests
├── strategies/           # Strategy tests
└── ...
```

Writing Tests

```
import pytest
from snapper.indicators.rsi import rsi

class TestRSI:
    """Test suite for RSI indicator."""

    def test_rsi_returns_series_with_correct_length(self) -> None:
        """
        Given a price series of length N
        When RSI is calculated with period P
        Then the result has length N.
        """
        prices = pd.Series([100, 102, 101, 103, 105])
        result = rsi(prices, period=14)
        assert len(result) == len(prices)
```

```
def test_rsi_values_in_valid_range(self) -> None:
    """
    Given any price series
    When RSI is calculated
    Then all values are between 0 and 100.
    """
    prices = pd.Series([100, 102, 101, 103, 105, 104, 106])
    result = rsi(prices, period=3)
    assert (result >= 0).all()
    assert (result <= 100).all()
```

Async Tests

```
import pytest

@pytest.mark.asyncio
async def test_fetch_candles() -> None:
    """Test fetching candles from repository."""
    repo = Repository(db_url)
    async with repo.session() as session:
        candles = await repo.get_candles(session, "BTC-USD", limit=10)
        assert len(candles) <= 10
```

Fixtures

```
@pytest.fixture
def sample_candles() -> list[Candle]:
    """Provide sample candle data for tests."""
    return [
        Candle(open=100, high=105, low=95, close=102, volume=1000),
        Candle(open=102, high=110, low=100, close=108, volume=1200),
    ]
```

100% Coverage

The project requires 100% code coverage (TDD). Check:

```
make cov
```

Configuration in .coveragerc.

Frontend

Setup

```
make ui-setup
```

Development

```
make ui-dev
```

Frontend at <http://localhost:3000/snapper/> with backend proxy.

Quality Checks

Command	Description
make ui-lint	ESLint
make ui-format	Prettier check
make ui-dead-code	Knip dead code
make ui-test	Vitest tests
make ui-cov	Tests with coverage

Fixes

```
make ui-fix    # lint-fix + format-fix + dead-code-fix
```

Type Generation

```
make ui-gen-types    # All types from OpenAPI + WebSocket
```

Database Migrations

Creating Migration

```
poetry run alembic revision -m "Add new column"
```

Applying

```
snapper db-upgrade
```

Rollback

```
snapper db-downgrade
```

Docker

Build

```
make docker-build
```

Run

```
make docker-run
```

docker-compose

```
docker-compose up -d
```

CI/CD

Pipeline executes:

1. make check — Quality checks
2. make cov — Tests with coverage
3. make ui-check — Frontend checks
4. make ui-cov — Frontend tests

Workflow

Before Commit

```
make check-all
```

Before PR

1. Ensure make check-all passes
2. Add/update tests
3. Update documentation if needed
4. Verify 100% coverage

Code Review Checklist

- [] Types for all functions
- [] Docstrings (Google style)
- [] No # comments
- [] Tests for new functionality
- [] 100% coverage
- [] make check-all passes

Project Structure

```
snapper/
├── src/snapper/           # Source code
│   ├── api/              # API schemas
│   ├── application/      # Business logic
│   ├── auth/             # Authentication
│   ├── cli/              # CLI interface
│   ├── config/           # Configuration
│   ├── core/             # Core types
│   ├── data/             # Data layer
│   ├── indicators/       # Technical indicators
│   ├── infrastructure/   # External integrations
│   ├── interface/        # WebSocket
│   ├── messaging/        # ZeroMQ
│   ├── server/           # FastAPI
│   ├── strategies/       # Trading strategies
│   └── utils/            # Utilities
├── tests/                # Tests
├── frontend/             # React dashboard
├── docs/                 # Documentation
├── scripts/              # Helper scripts
└── data/                 # Local data
```

Tools

Ruff

Linters and formatters. Configuration in pyproject.toml:

```
[tool.ruff.lint]
select = ["E", "F", "W", "I", "B", "UP", "C4", "SIM", "N", "PERF", "ISC",
"PLW", "PLC", "A", "TID", "D"]
```

Mypy

Type checker. Strict mode enabled:

```
[tool.mypy]
python_version = "3.12"
strict = true
```

Black

Formatter. Line length 100:

```
[tool.black]
line-length = 100
```

Pre-commit

Hooks run before commit:

```
pre-commit install
pre-commit run --all-files
```

Debugging

Logging

```
from loguru import logger

logger.info("Processing signal", signal=signal)
logger.debug("Calculated RSI", value=rsi_value)
logger.error("Failed to execute order", error=str(e))
```

ZMQ Logger

```
snapper zmq-logger --payload
```

Serial Tests

```
make test-serial
```

pdb/ipdb

```
import ipdb; ipdb.set_trace()
```

Documentation

Docstrings

Every module, class, and public function must have a docstring.

docs/

Markdown documentation in docs/ directory.

PDF

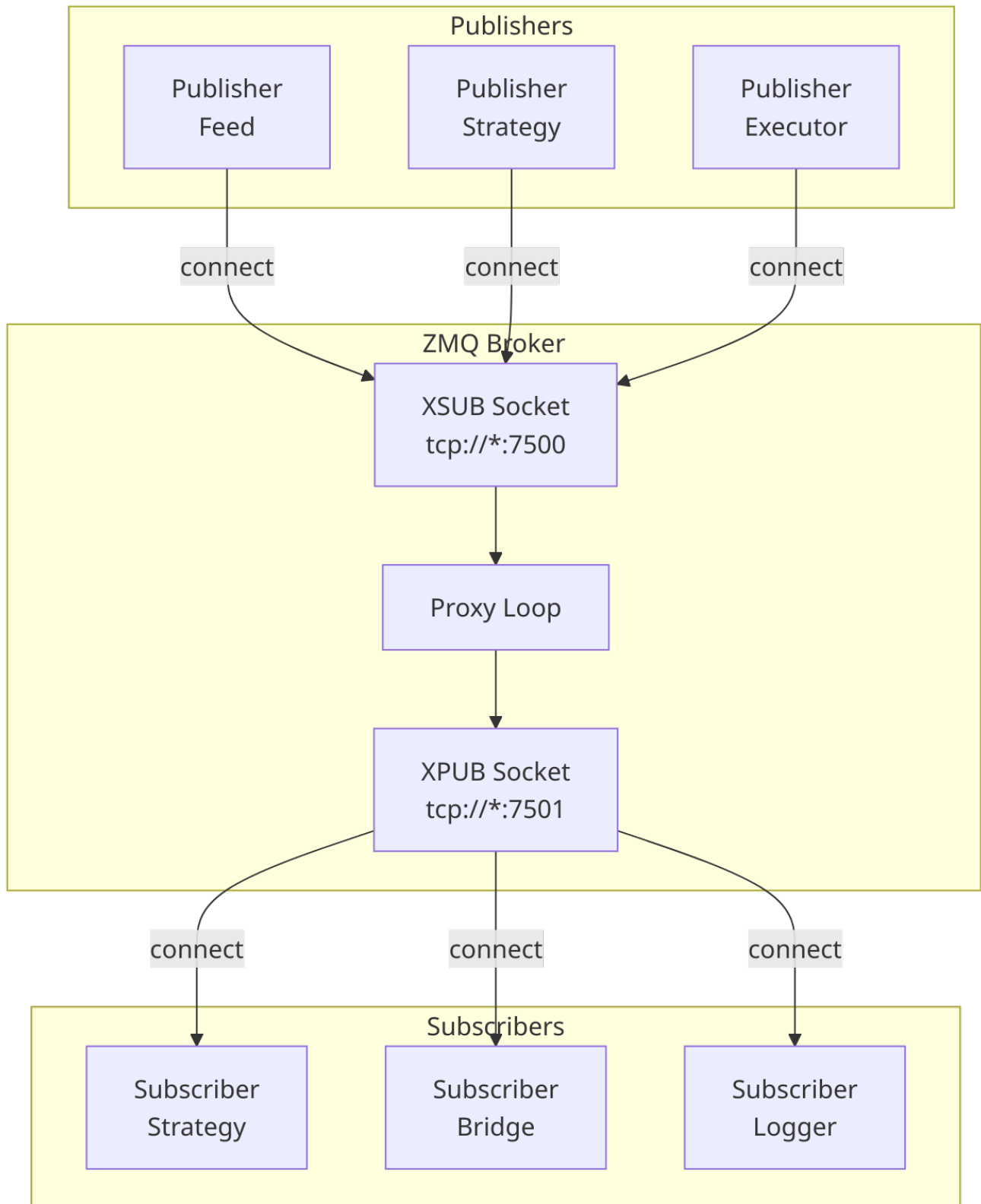
```
make docs-pdf
```

Generates snapper.pdf from README + docs/*.md.

Messaging (ZeroMQ)

Snapper uses ZeroMQ for inter-process communication in a pub/sub architecture. The central component is an XPUB/XSUB broker that routes messages.

Architecture



XPUB/XSUB Broker

The broker is the central message hub:

- **XSUB** (tcp://*:7500) — Publishers connect here
- **XPUB** (tcp://*:7501) — Subscribers connect here

The broker forwards messages in both directions:

- Data: XSUB → XPUB
- Subscriptions: XPUB → XSUB

Starting the Broker

```
snapper broker
```

Or programmatically:

```
from snapper.messaging.infrastructure.broker import ZmqBrokerThread

broker = ZmqBrokerThread()
broker.start()
# ...
broker.stop()
```

Topics

Topic format: {category}.{exchange}.{instrument}.{type}.{timeframe}

Market Data

Topic	Description
market.kraken.BTC-USD.candles.1h	Hourly BTC/USD candles from Kraken
market.kraken.BTC-USD.ticks	BTC/USD ticks from Kraken
market.polygon.AAPL.candles.1d	Daily AAPL candles from Polygon

Signals

Topic	Description
signals.paper.BTC-USD.rsi_btc_1h	Paper trading signals
signals.kraken.BTC-USD.live	Live signals from Kraken

Orders & Fills

Topic	Description
orders.kraken.BTC-USD	Order requests for Kraken
fills.kraken.BTC-USD	Order executions from Kraken
order_status.kraken.*	Order statuses

System

Topic	Description
system.heartbeat	Component heartbeats
system.settings.changed	Configuration changes
replay.start	Data replay start
replay.end	Replay end

Message Envelopes

All messages use typed envelopes (Pydantic).

TickEnvelope

```
from snapper.messaging.schemas.messages import TickEnvelope

tick = TickEnvelope(
    instrument="BTC-USD",
    exchange="kraken",
    price=42000.0,
    volume=0.5,
    timestamp=datetime.now(UTC),
)
```

Fields:

Field	Type	Description
type	string	"tick"
instrument	string	Instrument symbol

Field	Type	Description
exchange	string	Exchange name
price	float	Price
volume	float	Volume
timestamp	datetime	Timestamp

BarEnvelope

```
from snapper.messaging.schemas.messages import BarEnvelope

bar = BarEnvelope(
    instrument="BTC-USD",
    exchange="kraken",
    timeframe="1h",
    open=42000.0,
    high=42500.0,
    low=41800.0,
    close=42300.0,
    volume=1234.56,
    timestamp=datetime.now(UTC),
)
```

Fields:

Field	Type	Description
type	string	"bar"
instrument	string	Symbol
exchange	string	Exchange
timeframe	string	Timeframe (1m, 5m, 1h, etc.)
open	float	Open price
high	float	High
low	float	Low
close	float	Close price
volume	float	Volume
timestamp	datetime	Timestamp

SignalEnvelope

```
from snapper.messaging.schemas.messages import SignalEnvelope

signal = SignalEnvelope(
    instrument="BTC-USD",
    exchange="paper",
    side="buy",
    strength=0.85,
    reason="RSI <= 30",
    strategy_name="rsi_btc_1h",
    price=42000.0,
)
```

Fields:

Field	Type	Description
type	string	"signal"
instrument	string	Symbol
exchange	string	Target exchange
side	string	"buy" or "sell"
strength	float	Signal strength 0.0-1.0
reason	string	Reason
strategy_name	string	Strategy name
price	float	Price
metadata	dict	Additional data

OrderRequestEnvelope

```
from snapper.messaging.schemas.messages import OrderRequestEnvelope

order = OrderRequestEnvelope(
    instrument="BTC-USD",
    exchange="kraken",
    client_order_id="ord_123",
    side="buy",
    order_type="limit",
    price=42000.0,
    quantity=0.1,
)
```

FillEnvelope

```
from snapper.messaging.schemas.messages import FillEnvelope

fill = FillEnvelope(
    instrument="BTC-USD",
    exchange="kraken",
    order_id="ord_123",
    exchange_order_id="KRAKEN-456",
    side="buy",
    price=42000.0,
    quantity=0.1,
    fee=0.001,
    fee_asset="USD",
)
```

HeartbeatEnvelope

```
from snapper.messaging.schemas.messages import HeartbeatEnvelope

heartbeat = HeartbeatEnvelope(
    component="zmq_broker",
    status="healthy",
)
```

Publisher

Publishing messages via validated socket:

```
from snapper.messaging.infrastructure.validated_socket import ValidatedPublisher
from snapper.messaging.schemas.messages import BarEnvelope

async def publish_bars():
    publisher = ValidatedPublisher()
    await publisher.connect()

    topic = "market.kraken.BTC-USD.candles.1h"
    bar = BarEnvelope(
        instrument="BTC-USD",
        exchange="kraken",
        timeframe="1h",
        open=42000.0,
        high=42500.0,
        low=41800.0,
        close=42300.0,
        volume=1234.56,
    )

    await publisher.publish(topic, bar)
    await publisher.close()
```

Subscriber

Subscribing to messages:

```
from snapper.messaging.infrastructure.validated_socket import
ValidatedSubscriber
from snapper.messaging.schemas.messages import parse_message

async def subscribe_to_market():
    subscriber = ValidatedSubscriber()
    await subscriber.connect()
    await subscriber.subscribe("market.kraken.BTC-USD.")

    while True:
        topic, message = await subscriber.receive()
        envelope = parse_message(message)
        print(f"Received {envelope.type} on {topic}")
```

Market Data Publisher

Built-in publisher for exchange data:

```
snapper feed --symbols "BTC/USD,ETH/USD"
```

Programmatically:

```
from snapper.messaging.publishers.kraken import KrakenMarketDataPublisher

async def run_feed():
    publisher = KrakenMarketDataPublisher(symbols=["BTC/USD", "ETH/USD"])
    await publisher.start()
```

Order Executor

Service executing orders:

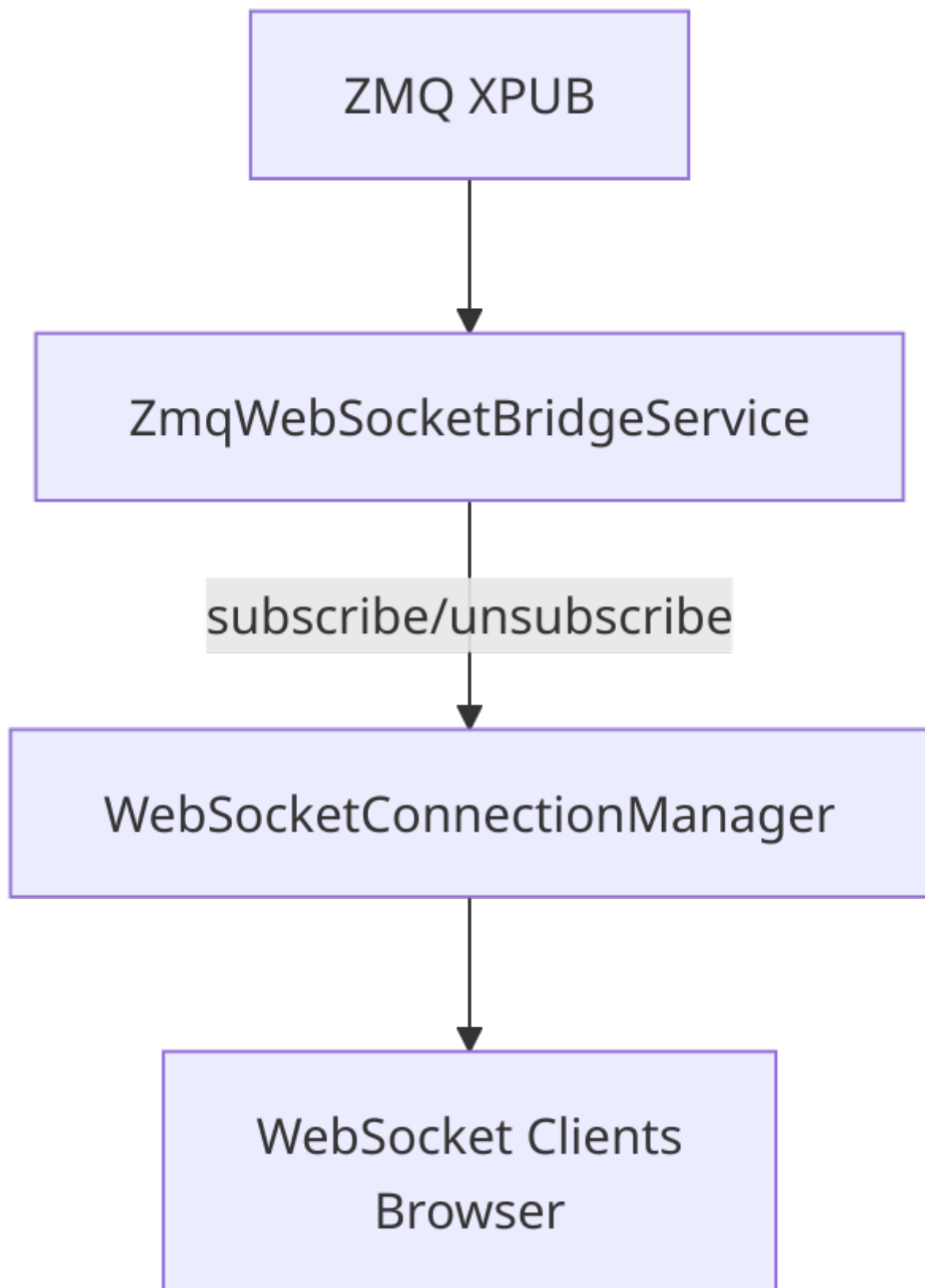
```
snapper executor -e kraken
```

Executor:

1. Subscribes to orders.{exchange}.* topics
2. Receives OrderRequestEnvelope
3. Executes order via exchange API
4. Publishes FillEnvelope or OrderStatusEnvelope

ZMQ-WebSocket Bridge

Bridge between ZMQ and WebSocket for frontend:



Bridge automatically:

- Subscribes to ZMQ topics when WebSocket client subscribes
- Forwards messages to WebSocket clients
- Unsubscribes when last client disconnects

Message Logger

Debug tool for monitoring ZMQ traffic:

```
snapper zmq-logger --payload --audit-file logs/zmq.jsonl
```

Logs all messages passing through the broker.

Configuration

Endpoints in .env:

```
ZMQ_BROKER_XSUB=tcp://127.0.0.1:7500  
ZMQ_BROKER_XPUB=tcp://127.0.0.1:7501
```

Throttling

TopicSchema defines throttling per topic:

```
TopicSchema(  
    name="market",  
    pattern="market.",  
    throttle_ms=100, # Max 10 msg/s  
    ...  
)
```

Message Validation

ValidatedPublisher/Subscriber validate schemas:

```
from snapper.messaging.topics.validation import validate_message_schema  
  
is_valid = validate_message_schema("market.kraken.BTC-USD.candles.1h", message)
```

Retry and Resilience

Sockets automatically:

- Reconnect on connection loss
- Buffer messages when broker unavailable
- LINGER=0 on close (discard pending)

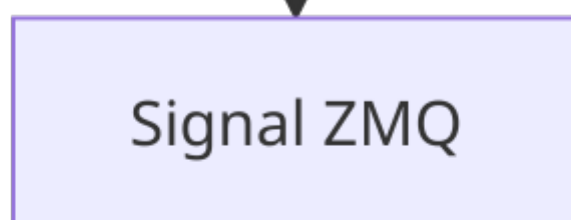
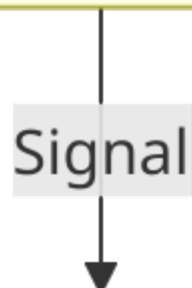
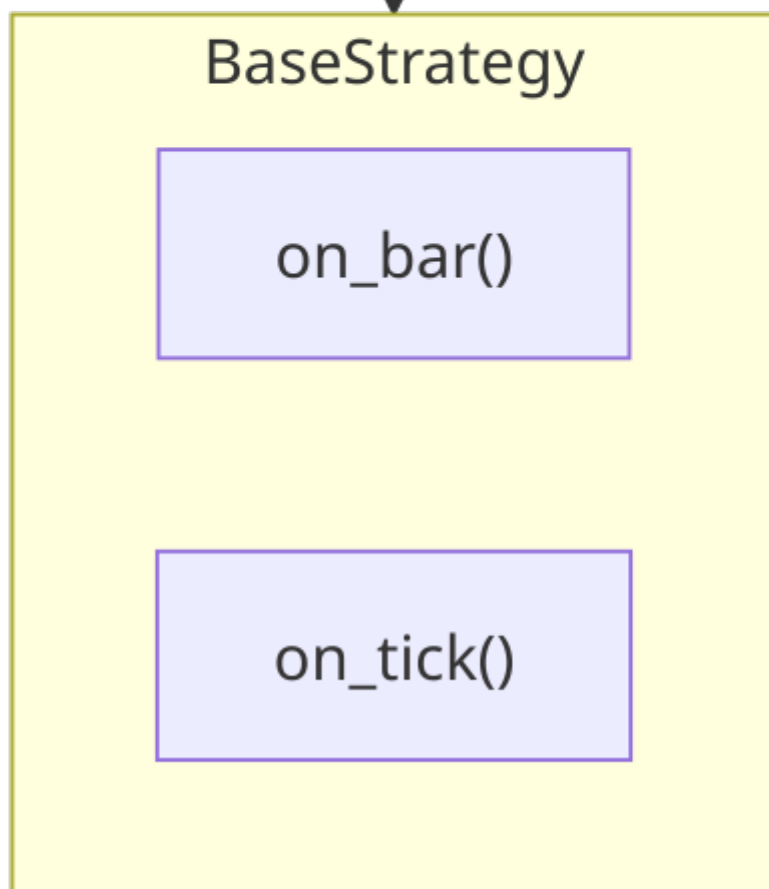
Best Practices

1. **One broker per system** — All components connect to the same broker
2. **Topic hierarchy** — Use hierarchy for filtering (market.kraken.*)
3. **Envelope types** — Always use typed envelopes
4. **Heartbeats** — Send heartbeats every 30s from components
5. **Graceful shutdown** — Close sockets with LINGER=0

Trading Strategies

Snapper provides a framework for creating trading strategies based on ZeroMQ messaging. Strategies subscribe to market data and publish signals.

Strategy Architecture



Creating Strategies

Basic Structure

```
from snapper.strategies.base import BaseStrategy, Signal, StrategyConfig
from snapper.strategies.decorators import register_strategy,
create_strategy_process
from snapper.messaging.schemas.messages import BarEnvelope

@register_strategy("MyStrategy")
@create_strategy_process(
    process_name="my_strategy_btc",
    default_config={
        "name": "my_strategy_btc",
        "inputs": ["market.kraken.BTC-USD.candles.1h"],
        "outputs": ["BTC-USD"],
        "exchange": "paper",
        "params": {
            "threshold": 0.5,
            "period": 14,
        },
    },
)
class MyStrategy(BaseStrategy):
    """Strategy description in docstring."""

    def __init__(self, config: StrategyConfig) -> None:
        """Initialize strategy."""
        super().__init__(config)
        self.threshold = self.params.get("threshold", 0.5)
        self.period = self.params.get("period", 14)

    async def on_bar(self, instrument: str, bar: BarEnvelope) -> Signal | None:
        """Process candle and generate signal."""
        # Access candle buffer
        candles = self.candle_buffer.get(instrument, [])
        if len(candles) < self.period:
            return None

        # Strategy logic
        closes = [c.close for c in candles]
        current_price = closes[-1]

        if self._should_buy(closes):
            return Signal(
                instrument=instrument,
                side="buy",
                strength=1.0,
                price=current_price,
                reason="Buy condition met",
                metadata={"custom_field": "value"},
            )

        if self._should_sell(closes):
```

```

        return Signal(
            instrument=instrument,
            side="sell",
            strength=1.0,
            price=current_price,
            reason="Sell condition met",
        )

    return None

def _should_buy(self, closes: list[float]) -> bool:
    """Buy condition logic."""
    return False

def _should_sell(self, closes: list[float]) -> bool:
    """Sell condition logic."""
    return False

async def reset(self) -> None:
    """Reset strategy state (for replay)."""
    pass

```

Decorators

@register_strategy(name)

Registers strategy class in factory under given name.

```

@register_strategy("RSIReversion")
class RSIReversion(BaseStrategy):
    ...

```

@create_strategy_process(process_name, default_config)

Creates process wrapper for strategy and registers in process manager.

```

@create_strategy_process(
    process_name="strategy_rsi_eth",
    default_config={
        "name": "rsi_eth_1h",
        "inputs": ["market.kraken.ETH-USD.candles.1h"],
        "outputs": ["ETH-USD"],
        "exchange": "paper",
        "params": {"period": 14, "upper": 70, "lower": 30},
    },
)
class RSIReversion(BaseStrategy):
    ...

```

Strategy Configuration

StrategyConfig

Field	Type	Description
name	string	Unique strategy instance name
strategy_class	string	Strategy class name
inputs	list[str]	List of ZMQ topics to subscribe
outputs	list[str]	List of instruments for signals
exchange	string	Target exchange (paper, kraken, zonda, walutomat)
params	dict	Strategy-specific parameters

Input Topics

Format: market.{exchange}.{instrument}.{type}.{timeframe}

Examples:

- market.kraken.BTC-USD.candles.1h — Hourly BTC/USD candles from Kraken
- market.kraken.ETH-USD.candles.15m — 15-minute ETH/USD candles
- market.polygon.AAPL.candles.1d — Daily AAPL candles from Polygon

Output Topics (Signals)

Generated automatically:

- Paper: signals.paper.{instrument}.{strategy_name}
- Live: signals.{exchange}.{instrument}.live

Signal Structure

```
@dataclass
class Signal:
    """Trading signal."""

    instrument: str          # Instrument symbol
    side: TradeSide          # "buy" or "sell"
    strength: float          # Signal strength 0.0-1.0
    reason: str              # Signal reason
    price: float             # Price at generation
```

```
timestamp: float      # Unix timestamp (optional)
metadata: dict         # Additional data
```

Built-in Strategies

RSIReversion

Mean-reversion strategy based on RSI.

Parameters:

Parameter	Default	Description
period	14	RSI period
upper	70.0	Overbought threshold (sell)
lower	30.0	Oversold threshold (buy)
cooldown	2	Candles between signals

Logic:

- Buy when RSI $\leq$ lower
- Sell when RSI $\geq$ upper

```
from snapper.strategies.rsi import RSIReversion
```

MACDCrossover

Trend-following strategy based on MACD.

Parameters:

Parameter	Default	Description
fast	12	Fast EMA
slow	26	Slow EMA
signal_period	9	Signal line period

Logic:

- Buy when MACD histogram crosses from negative to positive
- Sell when histogram crosses from positive to negative

```
from snapper.strategies.macd import MACDCrossover
```

Cointegration

Pairs trading strategy based on cointegration.

```
from snapper.strategies.cointegration import CointegrationStrategy
```

Data Access

Candle Buffer

```
async def on_bar(self, instrument: str, bar: BarEnvelope) -> Signal | None:
    candles = self.candle_buffer.get(instrument, [])

    # Last N candles
    recent = candles[-20:]

    # Close prices as pandas Series
    import pandas as pd
    closes = pd.Series([c.close for c in candles])
```

BarEnvelope

```
@dataclass
class BarEnvelope:
    open: float
    high: float
    low: float
    close: float
    volume: float
    timestamp: int
    timeframe: str
    instrument: str
```

Technical Indicators

RSI

```
from snapper.indicators.rsi import rsi
import pandas as pd

closes = pd.Series([c.close for c in candles])
```

```
rsi_values = rsi(closes, period=14)
current_rsi = rsi_values.iloc[-1]
```

MACD

```
from snapper.indicators.ta_lib_adapter import macd

macd_line, signal_line, histogram = macd(closes, fast=12, slow=26, signal=9)
```

TA-Lib (via adapter)

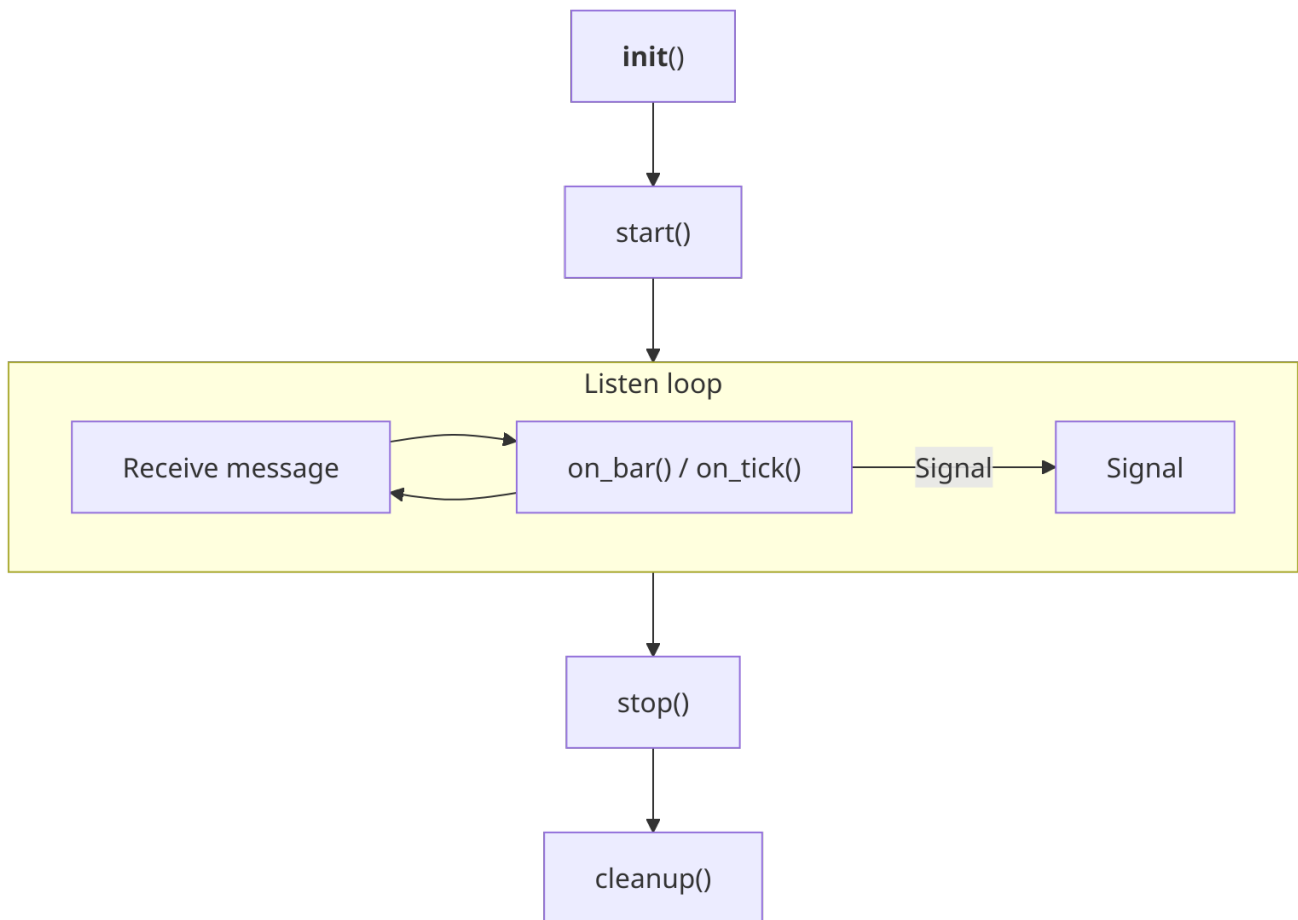
```
from snapper.indicators.ta_lib_adapter import rsi, macd, sma, ema
```

Configuration Validation

Framework automatically validates:

1. **Strategy name** — cannot be empty
2. **Inputs** — at least one required
3. **Outputs** — at least one instrument must be defined
4. **Exchange** — must be one of: paper, kraken, zonda, walutomat
5. **Instruments** — must be available on selected exchange
6. **Paper/live mixing** — mixing paper inputs with live exchange not allowed

Strategy Lifecycle



Running Strategies

Via Process Manager (recommended)

Strategies are automatically managed by the server:

```
snapper server
```

Strategies can be enabled/disabled in the dashboard.

Standalone (for testing)

```
import asyncio
from snapper.strategies.factory import StrategyFactory
from snapper.strategies.base import StrategyConfig

config = StrategyConfig(
    name="test_rsi",
    strategy_class="RSIReversion",
    inputs=["market.kraken.BTC-USD.candles.1h"],
```

```

        outputs=["BTC-USD"],
        exchange="paper",
        params={"period": 14, "upper": 70, "lower": 30},
    )

strategy = StrategyFactory.create("RSIReversion", config)

async def main():
    await strategy.start()

asyncio.run(main())

```

Testing Strategies

Unit Tests

```

import pytest
from snapper.strategies.rsi import RSIReversion
from snapper.strategies.base import StrategyConfig
from snapper.messaging.schemas.messages import BarEnvelope

@pytest.fixture
def rsi_strategy() -> RSIReversion:
    config = StrategyConfig(
        name="test_rsi",
        strategy_class="RSIReversion",
        inputs=["market.kraken.BTC-USD.candles.1h"],
        outputs=["BTC-USD"],
        exchange="paper",
        params={"period": 14, "upper": 70, "lower": 30},
    )
    return RSIReversion(config)

async def test_buy_signal_on_low_rsi(rsi_strategy: RSIReversion) -> None:
    # Prepare data with low RSI
    ...
    signal = await rsi_strategy.on_bar("BTC-USD", bar)
    assert signal is not None
    assert signal.side == "buy"

```

Backtesting

Use replay data from paper topics:

```

default_config={
    "inputs": ["replay.BTC-USD.candles.1h"],
    "exchange": "paper",
    ...
}

```